Spdt

PAGE No.:

DATE : / /

Q:1 Simple currency conversion web services

```
from http.server import BaseHTTPRequestHandler, HTTP
Server

class CurrencyService(BaseHTTPRequestHandler):

    def do_GET(self):

        amount = 8300
        usd = amount 183

        self.send_response(200)
        self.send_header("content-type", "text/plain")
        self.end_headers()

        message = f"INR {amount} = USD {usd:2f}"

        self.wfile.write(message.encode())

server = HTTPserver(("localhost", 8000), currencyservice)

print ("currency web service started")
print ("open: http://localhost: 8000")

server.serve_forever()
```

Aim: Perform a practical to create a simple web service for currency conversion applications.

Q:1 output

http://localhost:8000
You should Get:
INR 8300 = USD 100.00

Q:2 output

Currency Conversion
INR Amount: ₹8300.0
USD Amount: $ 100.00
Conversion Rate: 1 USD = ₹83

Spdt Practical NO:2

PAGE No.:

DATE : / /

Q:2 INR TO USD currency conversion

```
from http.server import BaseHTTPRequestHandler,HTTP
server
from urllib.parse import urlparse,parse_qs

class CurrencyConverter(BaseHTTPRequestHandler):

def do_GET(self):

  query =parse_qs(urlparse(self.path).query)

  amount =float(query.get("amount",[0])[0])
  result  = amount *83

  self.send_response(200)
  self.send_header("content-type","text/html")
  self.end_header()

  Output =f"""
  <html>
  <head>
    <title> Currency Converter. </title>
  </head>
  <body>
     <h1>currency Conversion</h1>
      <p>INR Amount: ₹ {amount} </p>
      <p> USD Amount: $ {result:2f} </p>
```

Spdt

PAGE No.: _____

DATE : __/__/__

```
    <p>Conversion Rate: 1 USD = ₹83</p>
</body>
</html>
"""

    self.wfile.write(output.encode())

server = HTTPServer(("localhost", 8001), CurrencyConverter)

print("Currency Converter started")
print("Open: http://localhost:8001?amount=8300")

server.serve_forever()
```

Aim: Perform a practical to implement currency conversion functionality using different units such as INR to USD.

Q:3 output

Response web service:

INR 8300 = USD 100.00

Result:

Thus, a web service was successfully consumed using a python client application

Spdt

PAGE No.:

DATE : / /

Q:3 Consume web services using a client application

```
from urllib.request import urlopen
url = "http://localhost:8000"

response = urlopen(url)

data = response.read().decode()

print("Response from web services:")
print(data)
```

Aim: Perform a practical to consume a web service using a client application.

Result:

Thus, a basic SOAP web service was successfully implemented.

Spdt

PAGE No.:

DATE : / /

Q:4 Basic SOAP web services

Aim: Peoform a practical to implement a basic SOAP web service.

```
from http.server import
BaseHTTPRequestHandler, HTTPServer

class SOAPService(BaseHTTPRequestHandler):
  def do_POST(self):

length = int(self.headers['Content-Length'])
data = self.rfile.read(length).decode()

print("Request received:")
print(data)

response = """<SOAP><message>Hello from SOAP
web service</message>
      </SOAP>
"""

self.send_response(200)
self.send_header("Content-type","text/XML")

self.end_header()

self.Wfile.write(response.encode())
```

Spdt

PAGE No.: _____

DATE : / /

```
server = HTTPserver(("localhost", 8002), SOAPservice
print("SOAP web service started")
print("Open :HTTP://localhost:8002")
```

CHARACTER . ABILITY . COMMITMENT
S.P. COLLEGE OF ARTS, COMMERCE AND SCIENCE

Q:5 output

SOAP Response:

<SOAP>

<message> Hello From SOAP web service </message>

</SOAP>

Result:

Thus, the SOAP web service was successfully tested using a client application.

Spdt

PAGE No.:

DATE: / /

Q:5 Test SOAP web service using a client

Aim: Perform a practical to test a SOAP web service using a client application.

```
import urllib.request
url = "http://localhost:8002"

SOAP_message = """
<SOAP>
    <message> Hello SOAP </message>
<SOAP>
"""

request = urllib.request.Request(url,
data = SOAP_message.encode(),
headers = {"content-type": "text/xml"},
method = "POST"
)

response =
urllib.request.urlopen(request)

result = response.read().decode()

print("SOAP Response:")
print(result)
```

Q:1 Output

File uploaded successfully!

Result:

Thus, a simple File upload web service was successfully developed using python.

Sundaram

Spdt

PAGE No.:

DATE : / /

Q:1 Develop an application for File upload using web services

Aim: To develop a simple web service application for uploading a File from a client to a server.

```
from http.server import HTTPServer, BaseHTTPRequestHandler
class UploadServer(BaseHTTPRequestHandler):

    def do_POST(self):
        length = int(self.headers['Content-Length'])

        data = self.rfile.read(length)

        with open("upload-file.txt", "wb") as file:
            file.write(data)

        self.send_response(200)
        self.end_headers()
        self.wfile.write(b"File uploaded successfully!")

server = HTTPServer(("localhost", 8000), UploadServer)

print("Upload server running at http://localhost:8000")
server.serve_forever()
```

Q:2 output

File downloaded successfully

Result:

Thus, a simple file download web service was successfully implemented

Sundaram
Books for Success...

Spdt

PAGE No.:

DATE : / /

Q:2 Develop an application for file Download using web services

Aim: To develop a simple web service that allows a client to download a file from a server.

```
from http.server import HTTPServer, BaseHTTPRequest
Handler
class DownloadServer(BaseHTTPRequestHandler):

def do_GET(self):

  try:
   with open("download.txt", "rb") as file:
     data = file.read()

     self.send_response(200)
     self.send_header("Content-Type", "application/octect-stream")
     self.send_header("Content-Length", str(len(data)))
     self.send_headers()

     self.wfile.write(data)

  except FileNotFoundError:
     self.send_response(404)
     self.end_headers()
     self.wfile.write(b"File not found")
```

Sundaram®
Books for Success...

Spdt

PAGE No.: _____

DATE : __/__/__

```
server = HTTPServer(("localhost", 8000), DownloadServer)
print("Download server running at http://localhost:8000")
server.serve_forever()
```

CHARACTER · ABILITY · CONFIDENCE
SPJMR INSTITUTE OF COLLEGE OF ARTS, COMMERCE AND SCIENCE

Q:3 Output:

The exact text may vary, but the output will look similar to

ID:1

Title: sunt aut facere repellat provident occaecati excepturi optio reprehenderit.

Body: quia et suscipit....

Result:

Thus, data was successfully retrieved from a cloud base REST API.

Sundaram
Books for Success...

PAGE No.:
DATE :

Spdt

Q:3 Explore cloud-Based APIs For Data Retrieval

Aim: To retrieve data from a cloud based APIs using python.

```
import urllib.request
import json

url = "https://jsonplaceholder.typicode.com/posts/1"

response = urllib.request.urlopen(url)

data = response.read().decode()

result = json.loads(data)

print("ID:", result["id"])
print("Title:", result["title"])
print("Body:", result["body"])
```

Q:4 output

```
POST ID:1
User ID:1
Title :sunt aut facere repellat provident occaecati
excepturi optio reprehenderit
POST ID:2
User ID:1
Title :qui est esse
```

Result:

Thus, data retrieved from a cloud-based API was successfully processed using python.

Sundaram®
Books for Success...

Spdt

PAGE No.:
DATE : / /

Q:4 Process Retrieved Data using Cloud-Based APIs

Aim: To retrieve JSON data from a cloud API and process the recieved information

```
import urllib.request
import JSON

URl = "hEEPs://jsonplaceholder.typicode.com/posts"

response =urlib.request.urlopen(URl)

data = response.read().decode()

posts = json.loads(data)

for post in posts[:5]:

    print("POST ID:", post["id"])
    print("User ID:",post["userId"])
    print("Title:",post["title"])
    print("-" * 40)
```

Spdt

PAGE No.:

DATE : / /

Q:5 Analyze Request-Response structure of REST and SOAP services

Aim: To Analyze the request-response structure of REST and SOAP web services.

Part A: REST Request

REST commonly uses HTTP methods such as:

GET
POST
PUT
DELETE

Example REST Request:

```
GET/posts/1 HTTP/1.1
Host: example.com
```

The server may return:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": 1,
  "name": "John",
  "age": 20
}
```

Spdt

PAGE No.: _____

DATE : __ / __ / ____

REST structure

CLIENT
↓ HTTP GET
REST server
↓ JSON
CLIENT

REST commonly uses lightweight formats such as:

JSON
XML

Part B: SOAP Request

SOAP uses XML message

Example SOAP request:

```
<SOAP-Envelope
 xmlns:SOAP="http://schemas.xmlsoap.org/soap/
envelope/">

 <SOAP:Header>
 </SOAP:Header>

 <SOAP:Body>

 <getstudent>
  <id>1</id>
```

Expected output:

Status code: 200
Response:

{'userId': 1, 'id': 1, 'title': '...', 'body': '...'}

Spdt

PAGE No.: _____

DATE : __ / __ / ____

Example SOAP response:

```
<SOAP:Envelope
xmlns:SOAP="http://schemas.xmlsoap.org/soap/
envelope/">

 <SOAP.Body>

 <getStudentResponse>

  </getstudentResponse>
</SOAP:Body>
</SOAP:Envelope>
```

Simple python REST Request Demonstration

```
import urllib.request
import json

url = "https://jsonplaceholder.typicode.com/posts/1

request = urllib.request.Request(
    url,
    method = "GET"
)
response = urllib.request.urlopen(request)
print ("status code:", response.status)
data = json.load(response.read().decode())
print ("Response:")
print (data)
```